HotelBay RESTFul API

A RESTFul API for a hotel management platform

Version 1.0

The HotelBay API presents itself as a RESTFul API for an online hotel management platform that allows hotels to manage rooms, reservations, and guests. The API should be implemented so that it can be consumed by user-friendly frontends such as Web Applications or Mobile Applications.

The platform should support multiple hotels operating independently while sharing the same infrastructure. Users should be able to search for accommodations, manage reservations, and interact with hotels through the platform. The system should ensure consistency between room availability, reservations, and payments, even when multiple users interact with the system simultaneously.

API operations

The API should consider the following operations, which represent the main functional areas of the platform. These operations describe the expected capabilities of the system at a conceptual level and should serve as a foundation for the design of the API resources, interactions, and data exchanges.

1. User management

The system should allow the management of platform users, including creation, update, consultation, and removal of users. The system must support two types of users:

1. Administrators, responsible for managing hotels and platform data;
2. Guests, who can search for rooms and make reservations.

Each user should have:

- Name
- Username
- Email address

Users should be uniquely identifiable within the system. Guests should only access and modify their own information, while administrators may manage data according to their permissions.

Users may have active reservations, past reservations, and submitted reviews associated with their account. Removing a user should not invalidate historical data already recorded in the system.

2. Hotel management

The system should allow administrators to manage hotels, including creation, update, consultation, activation, and deactivation of hotels. Each hotel should include:

- Hotel name
- Description
- Location
- Contact information
- Available services and amenities

Hotels may define operational policies such as:

- check-in and check-out times;
- cancellation policies;
- reservation constraints.

A hotel may be temporarily unavailable without being removed from the platform. Existing reservations must remain valid even if a hotel becomes inactive.

Each hotel manages its own rooms and reservations independently from other hotels.

3. Room management

The system should allow administrators to manage rooms belonging to a hotel. Each room should include:

- Room identifier or number
- Room type
- Description
- Capacity
- Price per night
- Availability status

Rooms belong to exactly one hotel. Room availability should depend both on administrative status and existing reservations. Rooms may be temporarily unavailable due to maintenance or operational reasons. Changes to room information should not invalidate reservations that were already confirmed.

4. Room category management

The system should allow management of room categories (e.g., single, double, suite, deluxe).It should be possible to:

- Create categories
- Update categories
- Delete categories

Categories may be hierarchical, allowing parent and subcategories. A room may belong to multiple categories simultaneously. Categories must exist before they can be associated with rooms. Removing a category should not compromise historical reservation information.

5. Room search and availability

The system should allow users to search for rooms across all hotels. Search filters should include:

- Hotel
- Location
- Room category
- Maximum and minimum price
- Number of guests
- Check-in and check-out dates

Users should be able to combine filters freely. Search results should reflect room availability considering existing reservations and room status. The system should handle situations where room availability changes between search and reservation attempts.

6. Reservation management

The system should allow guests to create reservations for available rooms. A reservation should include:

- Room
- Guest
- Check-in date
- Check-out date
- Number of guests

Reservations should have a lifecycle with states such as:

- pending
- confirmed
- canceled
- completed

The system must prevent overlapping confirmed reservations for the same room. Reservations in certain states may be modified or canceled according to platform rules and hotel policies. Reservation history should be preserved for auditing purposes.

7. Payment management

The system should allow payments associated with reservations. Each payment should include:

- Associated reservation
- Payment amount
- Payment date
- Payment status

A reservation may require payment before confirmation. Payment attempts may succeed or fail, and the system should maintain the history of payment operations. Changes in payment status should affect reservation state accordingly.

8. Reviews and ratings

After completing a stay, guests may submit reviews about the hotel. Each review should include:

- A textual description
- A rating score from 0 to 10

Only guests with completed reservations may submit reviews. A guest may review a hotel only once per reservation. Reviews should remain associated with both the hotel and the guest and may contribute to aggregated hotel ratings.

Data exchange format

Whenever possible, the API should use JSON-formatted data, appropriately structured for each situation.

The exchanged data should follow consistent naming conventions and clearly represent the resources managed by the system. The API should be designed to support extensibility without breaking existing clients.